

DATA-260 HOMEWORK 5 REPORT

Clinical Trial Listing Application

Student: Sanjana Thummalapalli

SID4: 7801

PORT_BASE: 8601

PREFIX: s7801

VERIFY_SEED: 267801

Domain: Clinical Trial Listings

Model: qwen3:4b

GitHub Repository: <https://github.com/sanjana-glitch-art/data260-7801>

INTRODUCTION

Homework 5 extended the existing clinical-trial listing application into a full-stack, database-backed, tool-enabled agent system. The project included relational database improvements, Redux frontend state management, Model Context Protocol servers, retry and fault-injection experiments, deterministic tool tests, and a local Ollama agent.

The application uses FastAPI for the backend, React for the frontend, Redux Toolkit for shared frontend state, MySQL for persistent storage, and Ollama with the qwen3:4b model for local agent reasoning.

2. RELATIONAL DATABASE AND REDUX FRONTEND

The existing MySQL database was expanded with a sponsors table. Sponsor information that had previously been repeated in clinical-trial records was normalized into this table.

The clinical_trials table was extended with the following fields:

- trial_code
- enrollment_target
- sponsor_id

The sponsor_id field connects every clinical trial to a sponsor through a foreign-key relationship. The completed database contained 5,000 linked clinical-trial records and nine sponsor records.

The backend provides CRUD operations for sponsors and clinical trials. Pydantic schemas validate request data, while SQLAlchemy manages database operations.

The React frontend was updated to use Redux Toolkit. The Redux store manages trial records, selected records, loading state, errors, and success messages. Asynchronous Redux actions retrieve, create, update, and delete records through the FastAPI backend.

Reusable components were created for navigation and clinical-trial forms. The completed frontend passed both the lint and production-build commands.

3. MODEL CONTEXT PROTOCOL SERVERS

Two MCP servers were implemented.

The MealDB MCP server demonstrates how MCP tools can retrieve information from an external public API. It provides four meal-related tools and was tested through MCP Inspector.

The clinical-trial MCP server exposes exactly three domain tools:

1. `search_trials` searches clinical trials using a text query.
2. `trial_details` retrieves a clinical trial using its numeric ID.
3. `trial_phase_summary` returns the number of trials in each phase and can optionally filter the results by sponsor.

Every clinical-trial tool returns the same structured response containing `ok`, `data`, and `error` fields. Invalid inputs return `ok` as `false`, `data` as `null`, and a descriptive error message. This consistent structure makes the tools easier for MCP clients and agents to process.

4. RETRIES AND FAULT INJECTION

A reliability wrapper was implemented to handle temporary failures. Each operation permits a maximum of three attempts and uses a timeout of three seconds.

The retry delays were:

- First retry: 0.025 seconds
- Second retry: 0.050 seconds

The delays follow an exponential-backoff pattern. The maximum-attempt rule prevents an operation from retrying indefinitely.

The deterministic fault-injection experiment used `VERIFY_SEED 267801`. It performed 50 calls at each of three failure rates, producing 150 total records.

At a 0% injected-failure rate, all 50 calls succeeded. No calls required retries. The mean latency was 8.872 milliseconds, and the p99 latency was 12.712 milliseconds.

At a 20% injected-failure rate, 49 of 50 calls succeeded, producing a 98% success rate. Seven calls required retries. The mean latency was 19.170 milliseconds, and the p99 latency was 84.758 milliseconds.

At a 50% injected-failure rate, 47 of 50 calls succeeded, producing a 94% success rate. Twenty-three calls required retries. The mean latency was 31.251 milliseconds, and the p99 latency was 90.977 milliseconds.

The experiment showed that retries recovered from most temporary failures. However, latency increased as the failure rate increased because calls required additional attempts and backoff delays.

5. TOOL ENTRY POINT AND OFFLINE TESTS

A shared `execute_tool` function was created. It receives a tool name and input values, executes the appropriate domain operation, and returns a JSON string using the standard response envelope.

The entry point supports `search_trials`, `trial_details`, and `trial_phase_summary`. Unknown tool names and invalid values produce controlled error responses rather than uncaught exceptions.

A privacy rule was also implemented. Requests that attempt to retrieve a clinical-trial submitter email address are blocked. Normal trial-detail results remove the submitter email before returning data.

Nine offline tests were implemented:

- Valid trial search
- Invalid trial search
- Valid trial-detail request
- Invalid trial-detail request
- Valid phase aggregation
- Invalid phase aggregation
- Submitter-email safety block
- Unknown-tool handling
- Agent maximum-step termination

All nine tests passed successfully.

6. LOCAL OLLAMA AGENT

A local agent was implemented using Ollama and the qwen3:4b model. The agent receives a user request, selects an appropriate tool, executes it, reviews the structured result, and generates a final answer.

The agent includes structured JSON decisions, domain-tool execution, tool-call counting, maximum-step enforcement, privacy enforcement, JSONL logging, and model token statistics.

Four agent scenarios were executed.

In the first scenario, the agent searched for sleep-related clinical trials. It completed in two steps with one tool call. The latency was 29,717.352 milliseconds.

In the second scenario, the agent retrieved the title, phase, sponsor, and enrollment target for trial ID 1. It completed in two steps with one tool call. The latency was 9,863.639 milliseconds.

In the third scenario, the agent processed a request for the number of trials in each phase. It completed in three steps with two tool calls. The latency was 11,625.893 milliseconds. This scenario demonstrated a correction loop and the importance of validating model-generated arguments.

In the fourth scenario, the user requested the submitter email for trial ID 1. The safety rule blocked the request. The scenario stopped after one step and one tool call with the stop reason `safety_rule_block`. The latency was 3,756.577 milliseconds.

The complete agent experiment used eight model turns, 2,282 input tokens, and 792 output tokens, resulting in 3,074 total tokens.

7. SAFETY AND EXECUTION CONTROL

The project does not depend on the language model alone for safety. Important controls are enforced directly by application code.

Submitter email addresses are removed from ordinary tool results. Explicit requests for submitter emails are blocked. Invalid parameters are rejected before database access, and unknown tool names cannot be executed.

The agent is restricted by a maximum number of steps. Retry attempts are also limited. Agent decisions and tool results are stored in JSONL logs, making each run observable and reviewable.

A deterministic MockModel test verified that the agent stops after reaching the configured maximum step count. The test stopped after two steps and two tool calls with the stop reason `max_steps`.

8. TECHNICAL CHALLENGES

The database migration initially failed because the original and new tables used different MySQL collations. The collations were aligned before completing the sponsor migration.

The React build initially failed because several Redux, API-client, and reusable-form modules were missing. These modules were created and connected to the main application.

The installed MCP 2.x package was incompatible with the FastMCP interface used by the implementation. A separate virtual environment with an MCP version below 2 was created for the MCP servers.

The Ollama model encountered memory-allocation errors on the available hardware. Other applications were stopped, and the context and prediction sizes were reduced. These changes allowed all four scenarios to run.

The tool entry point also required special handling because some domain operations returned both a response and execution metadata. The wrapper was updated to return only the standard response envelope to the agent.

9. RESULTS AND DISCUSSION

The completed system demonstrates that a reliable application agent requires more than model-generated output. The language model can select tools and summarize information, but application code must validate inputs, enforce privacy rules, limit retries, stop excessive execution, and maintain logs.

The fault-injection experiment demonstrated that retries improve reliability during temporary failures. However, retries also increase latency. Therefore, retry attempts should remain limited and should only be used for operations that are safe to repeat.

Redux improved frontend organization by placing application data, loading state, errors, selected records, and success messages in one predictable store.

Database normalization reduced repeated sponsor information and established a clear relationship between clinical trials and sponsors.

10. CONCLUSION

Homework 5 successfully extended the clinical-trial application with relational data, Redux state management, MCP servers, retry mechanisms, deterministic fault injection, offline tests, and a local Ollama agent.

The final implementation includes 5,000 linked clinical-trial records, nine sponsors, two MCP servers, three domain tools, 150 fault-injection records, nine passing offline tests, four agent scenarios, a privacy rule, maximum-step enforcement, and structured execution logs.

The project demonstrated how databases, frontend state management, tool protocols, reliability mechanisms, safety rules, and local language models can be combined into a controlled agent-based application.

11. REPRODUCIBILITY

The primary verification commands were:

```
python -m code.test_hw5_tools
```

```
python -m code.verify_hw5
```

The expected results are:

Final summary: 9/9 passed

PASSED: True

Machine-readable results are stored in reports/hw05/raw. The final verification result is stored in reports/hw05/verification.json.